Prague University of Economics and Business

Faculty of Informatics and Statistics



AI Tools Catalogue: A Knowledge Graph of AI Tools, Models and Providers

SEMESTRAL PROJECT

Course: 4IZ444

Authors: Ota Vlna, Matěj Fadrhons

Prague, December 2026

Declaration on the Use of Artificial Intelligence

We confirm that artificial intelligence tools **were used** in the preparation of this work, specifically in the following ways:

- improving grammar and language style,
- editing and extending parts of the text,
- generating parts of the source code (ontology, SPARQL queries, SHACL shapes and build scripts),
- generating data,
- other: drafting the text of the report.

Each use is documented separately in Appendix B, which explains in more detail which tool was used in which part of the work and highlights our own contribution.

We confirm that:

- our work is original and was prepared in accordance with ethical standards, in particular with the Code of Ethics of the Prague University of Economics and Business, all information sources used are cited and the generated content was verified by us;
- we accept full responsibility for the entire content of the work.

Abstrakt

Práce se zabývá návrhem a vytvořením znalostního grafu, který popisuje katalog nástrojů umělé inteligence – produkty jako ChatGPT, Claude, Gemini, Midjourney nebo GitHub Copilot, modely, na kterých tyto nástroje běží, jejich poskytovatele, podporované úlohy, vstupní a výstupní modality, licence, integrace, cenové plány a výsledky benchmarků. Znalostní graf má sloužit jako podpora při výběru AI nástroje pro vývojáře, IT oddělení a analytiky trhu.

Na základě deseti kompetenčních otázek byla navržena ontologie v jazyce OWL 2 s 13 třídami, 25 vlastnostmi a axiomy (hierarchie tříd, disjunktnost, funkční, inverzní, tranzitivní a symetrické vlastnosti). Ontologie odděluje nástroj od modelu a modeluje výsledky benchmarků jako n-ární vztah. Termíny byly sladěny s existujícími slovníky schema.org, W3C ML Schema, Dublin Core, FOAF a SKOS. Ontologie byla naplněna daty o 15 nástrojích, 27 modelech a 12 poskytovatelích (téměř 1 000 trojic), kompetenční otázky a pět dalších analytických dotazů byly převedeny do jazyka SPARQL a kvalita dat je kontrolována jedenácti tvary SHACL. Výsledky ukazují například silnou koncentraci trhu na úrovni modelů – nástroje třetích stran stojí převážně na modelech tří poskytovatelů.

Klíčová slova

znalostní graf, ontologie, OWL, SPARQL, SHACL, umělá inteligence, jazykové modely

Abstract

This work designs and builds a knowledge graph describing a catalogue of artificial intelligence tools – products such as ChatGPT, Claude, Gemini, Midjourney or GitHub Copilot – together with the models that power them, their providers, supported tasks, input and output modalities, licences, integrations, pricing plans and benchmark results. The knowledge graph is intended to support the selection of AI tools by developers, IT departments and market analysts.

Based on ten competency questions, an OWL 2 ontology with 13 classes, 25 properties and a range of axioms (class hierarchy, disjointness, functional, inverse, transitive and symmetric properties) was designed. The ontology separates tools from models and represents benchmark results as an n-ary relation; its terms are aligned with the existing vocabularies schema.org, W3C ML Schema, Dublin Core, FOAF and SKOS. The ontology was populated with data on 15 tools, 27 models and 12 providers (almost 1,000 triples), the competency questions and five additional analytical queries were translated into SPARQL, and data quality is checked by eleven SHACL shapes. The results show, for example, a strong concentration of the market at the model level – third-party tools rely mostly on the models of three providers.

Keywords

knowledge graph, ontology, OWL, SPARQL, SHACL, artificial intelligence, language models

Contents

Introduction....8
1 Domain description....9
1.1 Motivation and use case....9
1.2 Competency questions....9
2 Ontology design....11
2.1 Core concepts....11
2.2 Axioms and constraints....13
2.3 Conceptual model....14
2.4 Reuse of existing ontologies....15
3 Knowledge graph data....17
3.1 Data creation....17
3.2 Time scope of the snapshot....17
3.3 Statistics....18
4 SPARQL queries....20
4.1 CQ1: Coding tools available in Visual Studio Code....20
4.2 CQ2: Models with image input and text output....21
4.3 CQ3: Average price of paid plans per provider....22
4.4 CQ4: Best open-weights models on MMLU....23
4.5 CQ5: Number of tools per task....23
4.6 CQ6: Tools built on models of other providers....24
4.7 CQ7: Model lineage....25
4.8 CQ8: Largest context window per provider....26
4.9 CQ9: Providers offering both text and image generation....27
4.10 CQ10: Free tools running on openly licensed models....28
4.11 Q11: Best scores of providers compared with the benchmark average....29
4.12 Q12: Analysis of model names with string functions....30
4.13 Q13: Competitor network....31
4.14 Q14: Models used in tools but lacking benchmark results....33
4.15 Q15: Materialising tool dependencies with CONSTRUCT....33
5 SHACL validation....35
5.1 S1 AIToolShape....35
5.2 S2 AIModelShape....35

5.3 S3 LanguageModelShape and ImageGenerationModelShape....36
5.4 S4 BenchmarkResultShape....37
5.5 S5 PricingPlanShape....37
5.6 S6 ProviderShape....38
5.7 S7 LicenseShape....38
5.8 S8 BenchmarkShape....39
5.9 S9 VocabularyShape....39
5.10 S10 ModelLineageShape....40
5.11 Validation results....40
Conclusion....43
References....44
Appendices....I
Appendix A: Repository and reproducibility....I
Appendix B: Use of artificial intelligence tools....I
Appendix C: Contribution of team members....II

Introduction

Over the last three years the number of products based on artificial intelligence (AI) has grown very quickly. A single company typically offers several *tools* (OpenAI offers ChatGPT and Codex, Anthropic offers Claude and Claude Code), every tool can use several *models*, and the same model is often embedded in tools of completely different companies – GitHub Copilot, Cursor and Perplexity all let the user choose between models from OpenAI, Anthropic and Google. Each tool also has its own pricing tiers and integrations, and each model is described by its licence, modalities, context window and benchmark scores. This information is scattered over vendor pricing pages, model cards and leaderboards, which makes comparing tools laborious.

Such information is naturally a graph: tools, models, providers, licences and benchmarks are interlinked entities, and the interesting questions follow paths through them (for example "which free tools run on an openly licensed model that scores well on MMLU?"). Knowledge graphs represent exactly this kind of interlinked data and allow such questions to be answered with a single query (Hogan et al., 2021).

Goal. The goal of this work is to design and build a knowledge graph of AI tools that can answer practical selection and market-analysis questions, and to practise the whole process of designing, populating, querying and validating RDF data.

Method. We followed the competency-question-driven approach to ontology engineering (Grüninger & Fox, 1995; Noy & McGuinness, 2001) in these steps:

- definition of the use case and of ten competency questions (Chapter 1),
- identification of concepts, design of the ontology in OWL 2 and research of reusable vocabularies in Linked Open Vocabularies (Chapter 2),
- creation of the knowledge graph data in Turtle (Chapter 3),
- translation of the competency questions and five additional analytical questions into SPARQL and interpretation of the results (Chapter 4),
- definition of SHACL shapes and validation of the data, including intentionally invalid data (Chapter 5).

All steps are automated by a build script that parses all files, checks the documentation of the ontology, runs an OWL 2 RL reasoner, executes the queries and the validation, and inserts the results into this report, so that all numbers in the text correspond to the actual data. All artefacts are published in a public Git repository (Appendix A).

1 Domain description

1.1 Motivation and use case

The knowledge graph serves as a **catalogue and decision-support tool for selecting AI tools**. Its typical users are:

- **IT and procurement departments** comparing tools by price, integrations and licensing (for example “we need a coding assistant for VS Code and JetBrains for less than 25 USD per user”),
- **software developers** choosing a coding assistant or an open-weights model to run locally,
- **analysts and journalists** observing the market – which providers depend on whose models, how quickly model generations follow each other and how open-weights models compare with closed ones.

The knowledge graph is a manually curated snapshot of the market as of 30 September 2025; the reasons for this cut-off date are discussed in Section 3.2.

1.2 Competency questions

The knowledge graph must be able to answer the competency questions (CQ) listed in Table 1.1. Questions CQ3, CQ5, CQ6 and CQ8 have an analytical character (aggregation, comparison). Five further queries (Q11–Q15) exercise more advanced SPARQL features and are described in Chapter 4.

Table 1.1
Competency questions

ID	Competency question
CQ1	Which AI tools support code generation and can be used inside Visual Studio Code, and who provides them?
CQ2	Which AI models accept images as input and produce text, who develops them and in how many tools are they used?
CQ3	What is the average, minimum and maximum monthly price of paid plans per provider?
CQ4	Which openly licensed (open-weights) models perform best on MMLU, and how large are they?
CQ5	How many AI tools support each task, and which ones?
CQ6	Which AI tools are powered by models of a different provider than the one offering the tool?
CQ7	What is the complete lineage (all predecessors) of a given model and how far apart were the releases?
CQ8	Which language model has the largest context window for each provider?
CQ9	Which providers offer both text generation and image generation, and with which in-house image models?

ID	Competency question
CQ10	Which AI tools can be used for free and run on at least one openly licensed model?

2 Ontology design

2.1 Core concepts

From the competency questions we identified the core concepts of the domain. The central design decision is the separation of an **AI tool** (a product a user interacts with, such as ChatGPT) from an **AI model** (a trained model, such as GPT-5). Without this distinction questions CQ2, CQ6, CQ7 or CQ10 could not be expressed. Two concepts were added to the eight initially proposed ones:

- **BenchmarkResult** – a score is a property of the *pair* (model, benchmark), not of the model or the benchmark alone. It is therefore modelled as an n-ary relation, i.e. a separate node linking the model, the benchmark and the score, following the W3C pattern for n-ary relations (Noy & Rector, 2006) and the class `mls:ModelEvaluation` of the W3C ML Schema (Publio et al., 2018).
- **PricingPlan** – needed for price comparisons (CQ3, CQ10). A plan is a separate entity because one plan can cover several tools; for example the *Claude Pro* subscription covers both Claude and Claude Code, and *ChatGPT Plus* covers ChatGPT and Codex.

The ontology namespace is `https://otavlna.github.io/4iz444-ai-tools-kg/ontology#` (prefix `ait:`); instance data use `https://otavlna.github.io/4iz444-ai-tools-kg/data/` (prefix `aid:`). The 13 classes are listed in Table 2.1, the 15 object properties in Table 2.2 and the 10 datatype properties in Table 2.3.

Table 2.1
Classes of the ontology

Class	Superclass	Description
`ait:AITool`	`schema:SoftwareApplication`	End-user AI product (ChatGPT, Claude Code, Midjourney, …).
`ait:AIModel`	`mls:Model`	A specific trained model release (GPT-5, Claude Sonnet 4.5, …).
`ait:LanguageModel`	`ait:AIModel`	Model whose primary output is text or code.
`ait:ImageGenerationModel`	`ait:AIModel`	Model whose primary output is images.
`ait:Provider`	`schema:Organization`	Organisation developing models and/or offering tools.
`ait:Task`	– (close match `mls:Task`)	Kind of work: code generation, summarisation, …
`ait:Modality`	–	Data type consumed or produced: text, image, audio, video.

Class	Superclass	Description
ait:License	dcterms:LicenseDocument	Licence under which a model is available.
ait:OpenLicense	ait:License	Licence allowing download and use of the weights.
ait:Integration	–	Platform or interface of a tool (VS Code, Slack, REST API, …).
ait:Benchmark	schema:Dataset	Standardised evaluation (MMLU, SWE-bench Verified, …).
ait:BenchmarkResult	mls:ModelEvaluation	Score of one model on one benchmark (n-ary relation).
ait:PricingPlan	schema:Offer	Subscription tier with a monthly price.

Table 2.2
Object properties of the ontology

Property	Domain → Range	Characteristics	Description
ait:providedBy	AITool → Provider	functional; ⊑ schema:provider	Organisation that offers the tool.
ait:poweredBy	AITool → AIModel		Model used by the tool.
ait:developedBy	AIModel → Provider	inverse of develops; ⊑ schema:creator	Organisation that trained the model.
ait:develops	Provider → AIModel	inverse of developedBy	Models released by the provider.
ait:supportsTask	AITool → Task		Task the tool can be used for.
ait:inputModality	AIModel → Modality		Accepted input modality.
ait:outputModality	AIModel → Modality		Produced output modality.
ait:hasLicense	AIModel → License	⊑ dcterms:license	Licence of the model.
ait:integratesWith	AITool → Integration		Platform in which the tool is available.
ait:hasPricingPlan	AITool → PricingPlan		Plan under which the tool is offered.
ait:evaluatedModel	BenchmarkResult → AIModel	functional	Model the result belongs to.
ait:onBenchmark	BenchmarkResult → Benchmark	functional	Benchmark of the result.
ait:measuresTask	Benchmark → Task		Task the benchmark measures.
ait:derivedFrom	AIModel → AIModel	transitive	Successor or derivative of an earlier model.
ait:competesWith	AITool → AITool	symmetric, irreflexive	Direct market competitors.

Table 2.3
Datatype properties of the ontology

Property	Domain → Range	Characteristics	Description
`ait:releaseDate`	AITool → `xsd:date` AIModel	functional; ⊑ `dcterms:issued`	First public availability.
`ait:contextWindow`	LanguageModel → `xsd:integer`	functional	Maximum tokens per request.
`ait:parameterCount`	AIModel → `xsd:decimal`	functional	Parameters in billions (if published).
`ait:score`	BenchmarkResult → `xsd:decimal`	functional	Score in percent (0–100).
`ait:metric`	Benchmark → `xsd:string`	functional	Metric of the benchmark (accuracy, pass@1, …).
`ait:monthlyPrice`	PricingPlan → `xsd:decimal`	functional	Price per user per month; 0 means free.
`ait:currency`	PricingPlan → `xsd:string`	functional	ISO 4217 currency code.
`ait:foundedYear`	Provider → `xsd:gYear`	functional	Year the provider was founded.
`ait:headquartersCountry`	Provider → `xsd:string`		Country of the headquarters.
`ait:spdxId`	License → `xsd:string`	functional	SPDX licence identifier.

Names of all entities are given by `rdfs:label`, homepages by the reused `foaf:homepage`. Every class and property of the ontology has an `rdfs:label` and an `rdfs:comment`; this is checked automatically by the build script.

2.2 Axioms and constraints

Besides domain and range restrictions for all 25 properties, the ontology (W3C OWL Working Group, 2012) contains the following axioms:

- **class hierarchy** – `LanguageModel` ⊑ `AIModel`, `ImageGenerationModel` ⊑ `AIModel`, `OpenLicense` ⊑ `License`, and alignment of the main classes to external superclasses (schema.org, ML Schema, Dublin Core),
- **disjointness** – `owl:AllDisjointClasses` over the ten top-level classes (a tool cannot be a model, a provider cannot be a task, …) and `LanguageModel owl:disjointWith ImageGenerationModel`,
- **property characteristics** – functional properties (`providedBy`, `evaluatedModel`, `onBenchmark`, `releaseDate`, `score`, `monthlyPrice` and others), an inverse pair (`developedBy` / `develops`), a transitive property (`derivedFrom`, the lineage of a model) and a symmetric and irreflexive property (`competesWith`),

- **union domain** – `releaseDate` is applicable to both tools and models (`rdfs:domain [ owl:unionOf (ait:AITool ait:AIModel) ]`),
- **sub-properties** of reused vocabularies (`hasLicense` ⊑ `dcterms:license`, `releaseDate` ⊑ `dcterms:issued`, `providedBy` ⊑ `schema:provider`, `developedBy` ⊑ `schema:creator`).

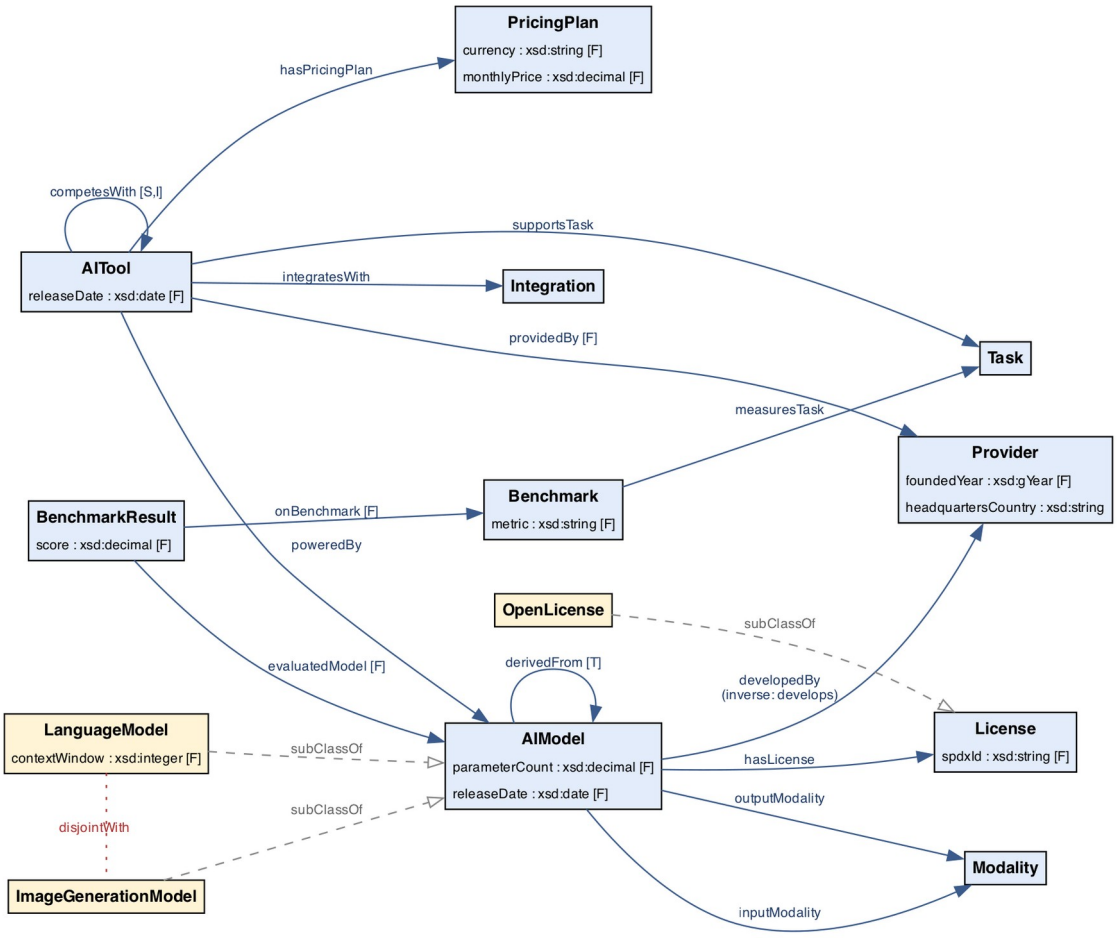
The property `derivedFrom` is only transitive, not irreflexive: OWL 2 DL does not allow irreflexivity on transitive (non-simple) properties. The rule "a model cannot be derived from a newer model" is therefore enforced by SHACL (shape S10 in Chapter 5).

The ontology is valid Turtle (Beckett et al., 2014); all reused external terms are explicitly declared so that it can be loaded into Protégé as OWL 2. It is free of logical inconsistencies: OWL 2 RL closure (owlrl 7.6.2) over ontology + data inferred 1657 additional triples; **0 inconsistencies** found (disjoint-class membership, `owl:Nothing`, irreflexive and functional property violations).

2.3 Conceptual model

The conceptual model of the ontology is shown in Figure 2.1. The diagram is generated automatically from the Turtle file, so it always corresponds to the actual ontology. The two central classes are `AITool` and `AIModel`, connected by `poweredBy`; `BenchmarkResult` and `PricingPlan` are the auxiliary nodes that carry numeric values for the analytical queries.

Figure 2.1
Conceptual model of the AI Tools Catalogue ontology



Note: Own work. Blue boxes are top-level classes, yellow boxes are subclasses (dashed arrows = rdfs:subClassOf). Solid arrows are object properties from domain to range; datatype properties are listed inside the classes. F = functional, T = transitive, S = symmetric, I = irreflexive.

2.4 Reuse of existing ontologies

We searched Linked Open Vocabularies (Vandenbussche et al., 2017), schema.org (Guha et al., 2016) and W3C specifications for terms that cover our concepts. The most relevant candidates are summarised in Table 2.4.

Table 2.4
Candidate vocabularies for reuse

Vocabulary	Relevant terms	Fit for our domain
schema.org	`SoftwareApplication`, `Organization`, `Offer`, `price`, `priceCurrency`, `provider`, `creator`, `Dataset`, `isBasedOn`, `sameAs`	Very good for products, organisations and offers; no notion of AI models, modalities or benchmarks.

Vocabulary	Relevant terms	Fit for our domain
W3C ML Schema (`mls:`)	`Model`, `ModelEvaluation`, `EvaluationMeasure`, `Task`, `Implementation`	Designed for machine-learning experiments; models and evaluations map well. Its `Task` means an ML task (classification, regression), which is only close to our user-facing task.
Dublin Core Terms	`license`, `LicenseDocument`, `issued`, `created`, `creator`, `title`	Standard metadata (DCMI Usage Board, 2020); ideal for licences and dates.
FOAF	`Organization`, `homepage`	Widely used (Brickley & Miller, 2014); `homepage` directly reusable.
SKOS	`Concept`, `ConceptScheme`, `broader`, `exactMatch`, `closeMatch`	Suitable for controlled lists (tasks, modalities) and alignments (Miles & Bechhofer, 2009).
SPDX License List	licence identifiers	Standard identifiers of licences such as Apache-2.0 or MIT (SPDX Workgroup, n.d.).
Wikidata	entities of providers (OpenAI Q21708200, Google Q95, Meta Q380, GitHub Q364)	Links of instances to the Linked Open Data cloud.

Decision. We define our **own core classes and properties** in the `ait:` namespace and **align** them with the external vocabularies instead of using the external terms directly, for three reasons:

- *Precise semantics and constraints.* We need domain and range restrictions, disjointness and functional properties (for example "a tool has exactly one provider"). Asserting such axioms on `schema:provider` or `mls:Model` would change the meaning of terms we do not own and would conflict with their intentionally loose definitions (schema.org uses `domainIncludes` and `rangeIncludes` instead of strict domains and ranges).
- *Missing concepts.* No vocabulary covers the distinction between a tool and a model, modalities, context windows or pricing plans shared by several products.
- *Interoperability is preserved* through `rdfs:subClassOf` (for example `ait:AITool` ⊑ `schema:SoftwareApplication`, `ait:BenchmarkResult` ⊑ `mls:ModelEvaluation`), `rdfs:subPropertyOf` (`ait:hasLicense` ⊑ `dcterms:license`, `ait:releaseDate` ⊑ `dcterms:issued`) and `skos:closeMatch` (`ait:Task` and `mls:Task`). An application that understands schema.org or Dublin Core can therefore still interpret our data after RDFS reasoning.

Some terms are **reused directly** because their semantics fits exactly and no additional constraints are needed: `rdfs:label` (names), `foaf:homepage` (homepages), `schema:sameAs` (links of providers to Wikidata), `schema:isBasedOn` (output of the CONSTRUCT query Q15), Dublin Core and VANN for the ontology metadata, and SPDX identifiers as values of `ait:spdxId`.

3 Knowledge graph data

3.1 Data creation

The knowledge graph (file `data/ai-tools-data.ttl`) was created manually with the help of a large language model, based on publicly available information (vendor pricing pages, model cards and release announcements), and then reviewed by the authors. It covers 15 tools from 12 providers and 27 models. Besides the tools named in the original topic (ChatGPT, Claude, Gemini, Midjourney, GitHub Copilot) it includes coding agents (Claude Code, OpenAI Codex, Cursor, Gemini Code Assist), an AI search engine (Perplexity), chat applications of open-weights providers (Meta AI, Le Chat, DeepSeek Chat, Qwen Chat) and a tool for running models locally (Ollama). Prices are list prices with monthly billing. Benchmark scores are mostly reported by the vendors themselves and were measured under different settings (number of shots, use of tools, reasoning mode), so they are indicative and serve to demonstrate analytical queries, not as an authoritative leaderboard.

3.2 Time scope of the snapshot

The data describe the state of the market as of **30 September 2025**; the newest model in the graph is Claude Sonnet 4.5, released on 29 September 2025. Models and price changes released after this date (late 2025 and 2026) are deliberately not included, for the following reasons:

- *Internal consistency*. Prices, model line-ups and benchmark results change every few weeks. A graph that mixes facts valid at different dates would produce misleading comparisons (for example a 2026 model compared with 2025 prices of its competitors). All facts in the graph therefore refer to a single cut-off date.
- *Verifiability*. The data were generated with the help of a language model whose reliable knowledge ends around this period, and were reviewed against public sources. Extending the graph to later releases without the same verification would risk invented values, which is worse than an explicitly dated snapshot.
- *Comparable benchmarks*. For newer models vendors increasingly report results on newer benchmarks, so their scores on the five benchmarks used here are often not available.
- *Purpose of the work*. The goal is to practise modelling, querying and validation; the structure of the graph does not depend on the snapshot.

Updating the graph is straightforward: a new model is described by about ten triples (see the example in Listing 3.1), the SHACL shapes check that the new data are complete and well-formed, and query Q14 lists models that are used by tools but still lack benchmark results.

3.3 Statistics

The size of the ontology and of the knowledge graph is summarised in Table 3.1 and the number of instances per class in Table 3.2. All classes and all properties except `ait:develops` are used in the data; `ait:develops` is the inverse of `ait:developedBy` and its values are inferred by the reasoner.

Table 3.1
Size of the ontology and of the knowledge graph

Metric	Value
Triples in the ontology	254
Triples in the knowledge graph (data)	987
Classes defined (`ait:`)	13
Object properties defined	15
Datatype properties defined	10
`ait:` properties used in data	24 of 25
Distinct subjects in data	170

Table 3.2
Number of instances per class (including instances of subclasses)

Class	Instances
`ait:BenchmarkResult`	44
`ait:PricingPlan`	34
`ait:AIModel`	27
`ait:LanguageModel`	22
`ait:AITool`	15
`ait:Integration`	13
`ait:Provider`	12
`ait:Task`	10
`ait:License`	6
`ait:OpenLicense`	5
`ait:Benchmark`	5
`ait:ImageGenerationModel`	5
`ait:Modality`	4

Listing 3.1 shows an example of the data – a tool, a model and a benchmark result.

Listing 3.1
Example triples of the knowledge graph (Turtle)

```
aid:GitHubCopilot a ait:AITool ;
    rdfs:label "GitHub Copilot" ;
    ait:providedBy aid:GitHub ;
    ait:releaseDate "2022-06-21"^^xsd:date ;
```

```
    ait:poweredBy aid:GPT41 , aid:GPT5 , aid:ClaudeSonnet45 , aid:Gemini25Pro ;
    ait:supportsTask aid:CodeGeneration , aid:CodeCompletion , aid:CodeReview ;
    ait:integratesWith aid:VSCode , aid:JetBrains , aid:GitHubPlatform , aid:CLI ;
    ait:hasPricingPlan aid:CopilotFree , aid:CopilotPro , aid:CopilotProPlus ,
aid:CopilotBusiness ;
    ait:competesWith aid:Cursor .

aid:ClaudeSonnet45 a ait:LanguageModel ;
    rdfs:label "Claude Sonnet 4.5" ;
    ait:developedBy aid:Anthropic ;
    ait:releaseDate "2025-09-29"^^xsd:date ;
    ait:contextWindow 200000 ;
    ait:inputModality aid:Text , aid:Image ;
    ait:outputModality aid:Text ;
    ait:hasLicense aid:Proprietary ;
    ait:derivedFrom aid:ClaudeSonnet4 .

aid:R_SWE_ClaudeSonnet45 a ait:BenchmarkResult ;
    ait:evaluatedModel aid:ClaudeSonnet45 ;
    ait:onBenchmark aid:SWEbenchVerified ;
    ait:score 77.2 .
```

A second file, `data/ai-tools-data-invalid.ttl`, contains eleven intentionally broken resources used to test the SHACL shapes (Section 5.11).

4 SPARQL queries

All queries (Harris & Seaborne, 2013) are stored in the directory `queries/` and were executed with RDFLib over the union of the ontology and the data; the ontology is needed for `rdfs:subClassOf*` paths. The results are inserted into this chapter automatically by the build script. The prefixes in Listing 4.1 are declared in every query file and are omitted from the listings for brevity.

Listing 4.1
Prefixes used in all queries

```
PREFIX ait:  <https://otavlna.github.io/4iz444-ai-tools-kg/ontology#>
PREFIX aid:  <https://otavlna.github.io/4iz444-ai-tools-kg/data/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
```

4.1 CQ1: Coding tools available in Visual Studio Code

The query (Listing 4.2) is a simple conjunctive query combining a task, an integration and a property path (`ait:providedBy/rdfs:label`) to the name of the provider.

Listing 4.2
SPARQL query CQ1

```
SELECT ?tool ?provider
WHERE {
  ?t a ait:AITool ;
     rdfs:label ?tool ;
     ait:supportsTask aid:CodeGeneration ;
     ait:integratesWith aid:VSCode ;
     ait:providedBy/rdfs:label ?provider .
}
ORDER BY ?tool
```

Table 4.1
Results of query CQ1

tool	provider
Claude Code	Anthropic
Gemini Code Assist	Google
GitHub Copilot	GitHub
OpenAI Codex	OpenAI

Note: The query returns 4 rows.

Four tools satisfy both conditions (Table 4.1). Cursor is not listed because it is a stand-alone editor (a desktop application) rather than a VS Code extension; chat assistants such as ChatGPT generate code but do not integrate into the IDE.

4.2 CQ2: Models with image input and text output

The class is matched with `a/rdfs:subClassOf* ait:AIModel`, so the query (Listing 4.3) works for instances of any subclass. `OPTIONAL` together with `COUNT(DISTINCT)` adds the number of tools using the model; it is 0 for models that are in the graph only as predecessors of newer models.

Listing 4.3
SPARQL query CQ2

```
SELECT ?model ?developer (COUNT(DISTINCT ?tool) AS ?usedInTools)
WHERE {
  ?m a/rdfs:subClassOf* ait:AIModel ;
     rdfs:label ?model ;
     ait:inputModality aid:Image ;
     ait:outputModality aid:Text ;
     ait:developedBy/rdfs:label ?developer .
  OPTIONAL { ?tool ait:poweredBy ?m }
}
GROUP BY ?model ?developer
ORDER BY DESC(?usedInTools) ?model
```

Table 4.2
Results of query CQ2

model	developer	usedInTools
Claude Sonnet 4.5	Anthropic	5
GPT-5	OpenAI	5
Gemini 2.5 Pro	Google	5
Claude Opus 4.1	Anthropic	2
Mistral Small 3.1	Mistral AI	2
GPT-4.1	OpenAI	1
GPT-4o	OpenAI	1
Gemini 2.5 Flash	Google	1
Gemma 3 27B	Google	1
Llama 4 Maverick	Meta	1
Mistral Medium 3	Mistral AI	1
o3	OpenAI	1
Claude 3.5 Sonnet	Anthropic	0
Claude 3.7 Sonnet	Anthropic	0
Claude Sonnet 4	Anthropic	0
Gemini 2.0 Flash	Google	0

Note: The query returns 16 rows.

Multimodal input has become the norm: 16 of the 22 language models accept images (Table 4.2). The most widely used ones are the flagships of the three large labs (Claude Sonnet 4.5, GPT-5, Gemini 2.5 Pro), each used in five tools – their own and those of third parties.

4.3 CQ3: Average price of paid plans per provider

Because one plan can be linked to several tools of the same provider, the inner sub-query (Listing 4.4) first selects `DISTINCT` (provider, plan) pairs; the outer query aggregates them with `COUNT`, `AVG`, `MIN` and `MAX` and rounds the average to two decimal places.

Listing 4.4
SPARQL query CQ3

```
SELECT ?provider ?currency (COUNT(?plan) AS ?paidPlans)
       (ROUND(AVG(?price) * 100) / 100 AS ?avgPrice)
       (MIN(?price) AS ?minPrice) (MAX(?price) AS ?maxPrice)
WHERE {
  {
    SELECT DISTINCT ?provider ?plan ?price ?currency
    WHERE {
      ?tool ait:providedBy/rdfs:label ?provider ;
            ait:hasPricingPlan ?plan .
      ?plan ait:monthlyPrice ?price ;
            ait:currency ?currency .
      FILTER (?price > 0)
    }
  }
}
GROUP BY ?provider ?currency
ORDER BY DESC(?avgPrice)
```

Table 4.3
Results of query CQ3

provider	currency	paidPlans	avgPrice	minPrice	maxPrice
Perplexity AI	USD	2	110.00	20.00	200.00
Google	USD	3	97.59	19.99	249.99
Anysphere	USD	3	86.67	20.00	200.00
OpenAI	USD	3	83.33	20.00	200.00
Midjourney, Inc.	USD	4	55.00	10.00	120.00
Anthropic	USD	3	50.00	20.00	100.00
GitHub	USD	3	22.67	10.00	39.00
Mistral AI	EUR	1	14.99	14.99	14.99

Note: The query returns 8 rows.

The averages in Table 4.3 are dominated by premium tiers: Perplexity, Google, Anysphere (Cursor) and OpenAI each have a tier for 200–250 USD, whereas the entry-level paid tier costs almost universally 20 USD. GitHub has the lowest average price of paid plans (22.67 USD) and shares the cheapest entry tier (10 USD) with Midjourney. Le Chat is priced in EUR, which is why the currency is part of the grouping – averaging prices in different currencies would be meaningless.

4.4 CQ4: Best open-weights models on MMLU

The licence is matched by its class (`a ait:OpenLicense`) and the score is obtained through the n-ary `BenchmarkResult` node (Listing 4.5).

Listing 4.5
SPARQL query CQ4

```
SELECT ?model ?license ?parametersB ?score
WHERE {
  ?m ait:hasLicense ?l ;
     rdfs:label ?model .
  ?l a ait:OpenLicense ;
     rdfs:label ?license .
  OPTIONAL { ?m ait:parameterCount ?parametersB }
  ?r ait:evaluatedModel ?m ;
     ait:onBenchmark aid:MMLU ;
     ait:score ?score .
}
ORDER BY DESC(?score)
```

Table 4.4
Results of query CQ4

model	license	parametersB	score
DeepSeek-R1	MIT License	671.0	90.8
gpt-oss-120b	Apache License 2.0	117.0	90.0
Llama 3.1 405B	Llama 3.1 Community License	405.0	88.6
DeepSeek-V3	MIT License	671.0	88.5
Qwen3-235B-A22B	Apache License 2.0	235.0	87.8
Llama 4 Maverick	Llama 4 Community License	400.0	85.5
Mistral Small 3.1	Apache License 2.0	24.0	80.6
Gemma 3 27B	Gemma Terms of Use	27.0	78.6

Note: The query returns 8 rows.

Openly licensed models reach the level of closed models on MMLU (Table 4.4; for comparison, GPT-4.1 scores 90.2 and GPT-4o 88.7). Model size helps but is not decisive: the 117-billion- parameter gpt-oss-120b scores almost as well as the 671-billion-parameter DeepSeek-R1, and the small Mistral Small 3.1 (24B) and Gemma 3 (27B) are only about 10 points behind models more than fifteen times larger.

4.5 CQ5: Number of tools per task

The query (Listing 4.6) groups the tools by task with `COUNT` and `GROUP_CONCAT`; the sub-query with `ORDER BY` makes the concatenated lists alphabetical and the results reproducible.

Listing 4.6
SPARQL query CQ5

```
SELECT ?task (COUNT(?t) AS ?toolCount)
       (GROUP_CONCAT(?tool; separator=", ") AS ?tools)
WHERE {
  {
    SELECT ?task ?t ?tool
    WHERE {
      ?t ait:supportsTask ?tk ;
         rdfs:label ?tool .
      ?tk rdfs:label ?task .
    }
    ORDER BY ?tool
  }
}
GROUP BY ?task
ORDER BY DESC(?toolCount) ?task
```

Table 4.5
Results of query CQ5

task	toolCount	tools
Code generation	12	ChatGPT, Claude, Claude Code, Cursor, DeepSeek Chat, Gemini, Gemini Code Assist, GitHub Copilot, Le Chat, Ollama, OpenAI Codex, Qwen Chat
Conversation	9	ChatGPT, Claude, DeepSeek Chat, Gemini, Le Chat, Meta AI, Ollama, Perplexity, Qwen Chat
Text generation	8	ChatGPT, Claude, DeepSeek Chat, Gemini, Le Chat, Meta AI, Ollama, Qwen Chat
Web search	6	ChatGPT, Claude, DeepSeek Chat, Gemini, Le Chat, Perplexity
Code review	5	Claude Code, Cursor, Gemini Code Assist, GitHub Copilot, OpenAI Codex
Summarization	5	ChatGPT, Claude, Gemini, Le Chat, Perplexity
Image generation	4	ChatGPT, Gemini, Meta AI, Midjourney
Code completion	3	Cursor, Gemini Code Assist, GitHub Copilot
Data analysis	2	ChatGPT, Claude

Note: The query returns 9 rows.

Code generation is the most common capability – 12 of the 15 tools support it (Table 4.5), because practically every general-purpose assistant can write code. Only specialised coding tools offer code review and inline code completion. The task *Knowledge and reasoning* does not appear, because it is used only to classify benchmarks, not tools.

4.6 CQ6: Tools built on models of other providers

The key condition of the query (Listing 4.7) is `FILTER (?tp != ?mp)`: the provider of the tool differs from the developer of the model.

Listing 4.7
SPARQL query CQ6

```
SELECT ?tool ?toolProvider
       (COUNT(DISTINCT ?m) AS ?externalModels)
```

```
       (GROUP_CONCAT(DISTINCT ?modelProvider; separator=", ") AS ?dependsOn)
WHERE {
  {
    SELECT ?tool ?toolProvider ?m ?modelProvider
    WHERE {
      ?t a ait:AITool ;
         rdfs:label ?tool ;
         ait:providedBy ?tp ;
         ait:poweredBy ?m .
      ?tp rdfs:label ?toolProvider .
      ?m ait:developedBy ?mp .
      ?mp rdfs:label ?modelProvider .
      FILTER (?tp != ?mp)
    }
    ORDER BY ?modelProvider
  }
}
GROUP BY ?tool ?toolProvider
ORDER BY DESC(?externalModels) ?tool
```

Table 4.6
Results of query CQ6

tool	toolProvider	externalModels	dependsOn
Ollama	Ollama	6	Alibaba Cloud, DeepSeek, Google, Meta, Mistral AI, OpenAI
GitHub Copilot	GitHub	4	Anthropic, Google, OpenAI
Cursor	Anysphere	3	Anthropic, Google, OpenAI
Perplexity	Perplexity AI	3	Anthropic, Google, OpenAI

Note: The query returns 4 rows.

Four tools have no in-house models at all (Table 4.6). GitHub Copilot, Cursor and Perplexity all depend on the same three labs (OpenAI, Anthropic, Google), which illustrates how concentrated the model layer of the market is. Ollama, a tool for running models locally, depends on six providers, all of which publish open weights.

4.7 CQ7: Model lineage

The transitive relation `ait:derivedFrom` is followed with the property path `ait:derivedFrom+` (Listing 4.8). The difference of two `xsd:date` values is an `xsd:duration` (for example `P130D`), from which the number of days is extracted with `REPLACE` and cast to an integer.

Listing 4.8
SPARQL query CQ7

```
SELECT ?model ?ancestor ?ancestorReleased ?daysBefore
WHERE {
  VALUES ?m { aid:ClaudeSonnet45 aid:GPT5 }
  ?m rdfs:label ?model ;
     ait:releaseDate ?released ;
     ait:derivedFrom+ ?a .
  ?a rdfs:label ?ancestor ;
     ait:releaseDate ?ancestorReleased .
  BIND (?released - ?ancestorReleased AS ?diff)
  BIND (xsd:integer(REPLACE(STR(?diff), "^P([0-9]+)D$", "$1")) AS ?daysBefore)
}
ORDER BY ?model ?ancestorReleased
```

Table 4.7
Results of query CQ7

model	ancestor	ancestorReleased	daysBefore
Claude Sonnet 4.5	Claude 3.5 Sonnet	2024-06-20	466
Claude Sonnet 4.5	Claude 3.7 Sonnet	2025-02-24	217
Claude Sonnet 4.5	Claude Sonnet 4	2025-05-22	130
GPT-5	GPT-40	2024-05-13	451
GPT-5	GPT-4.1	2025-04-14	115
GPT-5	03	2025-04-16	113

Note: The query returns 6 rows.

The data assert only the direct predecessors (Sonnet 4.5 → Sonnet 4 → 3.7 Sonnet → 3.5 Sonnet), but the property path returns the whole chain (Table 4.7). In both lineages the latest model followed its direct predecessor after 113–130 days; the earlier steps were longer (for example 249 days between Claude 3.5 Sonnet and 3.7 Sonnet). Two short lineages are not enough to generalise about release cycles – the query only shows how such an analysis can be done once the graph contains more model generations.

4.8 CQ8: Largest context window per provider

A sub-query computes the maximum context window per provider (Listing 4.9); it is joined back to the models to find which model or models reach it, and ties are aggregated with `GROUP_CONCAT`.

Listing 4.9
SPARQL query CQ8

```
SELECT ?provider ?contextWindow (COUNT(?m) AS ?modelCount)
       (GROUP_CONCAT(?model; separator=", ") AS ?models)
WHERE {
  {
    SELECT ?p (MAX(?cw) AS ?contextWindow)
    WHERE {
      ?x a ait:LanguageModel ;
         ait:developedBy ?p ;
         ait:contextWindow ?cw .
    }
    GROUP BY ?p
  }
  {
    SELECT ?p ?m ?model ?contextWindow
    WHERE {
      ?m a ait:LanguageModel ;
         ait:developedBy ?p ;
         ait:contextWindow ?contextWindow ;
         rdfs:label ?model .
    }
    ORDER BY ?model
  }
  ?p rdfs:label ?provider .
}
GROUP BY ?provider ?contextWindow
ORDER BY DESC(?contextWindow) ?provider
```

Table 4.8
Results of query CQ8

provider	contextWindow	modelCount	models
Google	1048576	3	Gemini 2.0 Flash, Gemini 2.5 Flash, Gemini 2.5 Pro
OpenAI	1047576	1	GPT-4.1
Meta	1000000	1	Llama 4 Maverick
Mistral AI	256000	1	Codestral 25.01
Anthropic	200000	5	Claude 3.5 Sonnet, Claude 3.7 Sonnet, Claude Opus 4.1, Claude Sonnet 4, Claude Sonnet 4.5
Alibaba Cloud	131072	1	Qwen3-235B-A22B
DeepSeek	128000	2	DeepSeek-R1, DeepSeek-V3

Note: The query returns 7 rows.

Google, OpenAI (GPT-4.1) and Meta (Llama 4 Maverick) offer context windows of about one million tokens (Table 4.8). All Anthropic models in the graph share the same 200,000-token window, so five models tie for the maximum.

4.9 CQ9: Providers offering both text and image generation

Two independent patterns for tools of the same provider (`?t1`, `?t2`) express that the provider offers both tasks; an `OPTIONAL` block looks for an image model developed by the same provider (Listing 4.10).

Listing 4.10
SPARQL query CQ9

```
SELECT ?provider
       (GROUP_CONCAT(DISTINCT ?imageModel; separator=", ") AS ?imageModels)
WHERE {
  ?p a ait:Provider ;
     rdfs:label ?provider .
  ?t1 ait:providedBy ?p ; ait:supportsTask aid:TextGeneration .
  ?t2 ait:providedBy ?p ; ait:supportsTask aid:ImageGeneration .
  OPTIONAL {
    ?t2 ait:poweredBy ?im .
    ?im a ait:ImageGenerationModel ;
        ait:developedBy ?p ;
        rdfs:label ?imName .
  }
  BIND (COALESCE(?imName, "(no in-house image model)") AS ?imageModel)
}
GROUP BY ?provider
ORDER BY ?provider
```

Table 4.9
Results of query CQ9

provider	imageModels
Google	Imagen 4
Meta	(no in-house image model)

provider	imageModels
OpenAI	gpt-image-1

Note: The query returns 3 rows.

OpenAI and Google cover both tasks with their own image models, gpt-image-1 and Imagen 4 (Table 4.9). Meta AI also generates images, but no Meta image model is part of the graph – the query reveals a gap in the data that could be filled in a future version. Midjourney is not listed because it only generates images.

4.10 CQ10: Free tools running on openly licensed models

The query (Listing 4.11) joins tools, pricing plans with a price of 0 and models whose licence is an instance of `ait:OpenLicense`.

Listing 4.11
SPARQL query CQ10

```
SELECT ?tool ?freePlan (COUNT(?m) AS ?openModelCount)
       (GROUP_CONCAT(?model; separator=", ") AS ?openModels)
WHERE {
  {
    SELECT DISTINCT ?tool ?freePlan ?m ?model
    WHERE {
      ?t a ait:AITool ;
         rdfs:label ?tool ;
         ait:hasPricingPlan ?plan ;
         ait:poweredBy ?m .
      ?plan ait:monthlyPrice 0.00 ;
            rdfs:label ?freePlan .
      ?m ait:hasLicense/a ait:OpenLicense ;
         rdfs:label ?model .
    }
    ORDER BY ?model
  }
}
GROUP BY ?tool ?freePlan
ORDER BY DESC(?openModelCount) ?tool
```

Table 4.10
Results of query CQ10

tool	freePlan	openModelCount	openModels
Ollama	Ollama (local, free)	6	DeepSeek-R1, Gemma 3 27B, Llama 3.1 405B, Mistral Small 3.1, Qwen3-235B-A22B, gpt-oss-120b
DeepSeek Chat	DeepSeek Chat (free)	2	DeepSeek-R1, DeepSeek-V3
Le Chat	Le Chat Free	1	Mistral Small 3.1
Meta AI	Meta AI (free)	1	Llama 4 Maverick
Qwen Chat	Qwen Chat (free)	1	Qwen3-235B-A22B

Note: The query returns 5 rows.

Five tools can be used free of charge with an open-weights model (Table 4.10). Ollama stands out with six such models because it runs them locally on the user's hardware;

DeepSeek Chat, Meta AI and Qwen Chat are hosted chat applications of providers that publish the weights of their models.

4.11 Q11: Best scores of providers compared with the benchmark average

This analytical query (Listing 4.12) has two levels of aggregation: the sub-query computes the average score per benchmark (only benchmarks with at least five results are considered) and the outer query takes the best model of each provider (`MAX`) and computes its difference from the average in percentage points.

Listing 4.12
SPARQL query Q11

```
SELECT ?benchmark ?provider (MAX(?score) AS ?bestScore) ?benchmarkAvg
       (ROUND((MAX(?score) - ?benchmarkAvg) * 10) / 10 AS ?diffToAvg)
WHERE {
  {
    SELECT ?b (ROUND(AVG(?s) * 10) / 10 AS ?benchmarkAvg) (COUNT(?r2) AS ?n)
    WHERE { ?r2 ait:onBenchmark ?b ; ait:score ?s . }
    GROUP BY ?b
  }
  ?r ait:onBenchmark ?b ;
     ait:score ?score ;
     ait:evaluatedModel/ait:developedBy/rdfs:label ?provider .
  ?b rdfs:label ?benchmark .
  FILTER (?n >= 5)
}
GROUP BY ?benchmark ?provider ?benchmarkAvg
ORDER BY ?benchmark DESC(?bestScore)
```

Table 4.11
Results of query Q11

benchmark	provider	bestScore	benchmarkAvg	diffToAvg
AIME 2025	OpenAI	94.6	88.5	6.1
AIME 2025	Anthropic	87.0	88.5	-1.5
AIME 2025	Google	86.7	88.5	-1.8
AIME 2025	Alibaba Cloud	81.5	88.5	-7.0
GPQA Diamond	Google	86.4	76.7	9.7
GPQA Diamond	OpenAI	85.7	76.7	9.0
GPQA Diamond	Anthropic	83.4	76.7	6.7
GPQA Diamond	DeepSeek	71.5	76.7	-5.2
GPQA Diamond	Alibaba Cloud	71.1	76.7	-5.6
GPQA Diamond	Meta	69.8	76.7	-6.9
HumanEval	Anthropic	92.0	89.2	2.8
HumanEval	OpenAI	90.2	89.2	1.0
HumanEval	Meta	89.0	89.2	-0.2
HumanEval	Mistral AI	88.4	89.2	-0.8

benchmark	provider	bestScore	benchmarkAvg	diffToAvg
MMLU	DeepSeek	90.8	87.1	3.7
MMLU	OpenAI	90.2	87.1	3.1
MMLU	Anthropic	88.7	87.1	1.6
MMLU	Meta	88.6	87.1	1.5
MMLU	Alibaba Cloud	87.8	87.1	0.7
MMLU	Mistral AI	80.6	87.1	-6.5
MMLU	Google	78.6	87.1	-8.5
SWE-bench Verified	Anthropic	77.2	64.5	12.7
SWE-bench Verified	OpenAI	74.9	64.5	10.4
SWE-bench Verified	Google	63.8	64.5	-0.7
SWE-bench Verified	DeepSeek	49.2	64.5	-15.3

Note: The query returns 25 rows.

The spread is largest on the agentic coding benchmark SWE-bench Verified (Table 4.11), where the best Anthropic model is 12.7 points above the average and DeepSeek 15.3 points below it. On the older MMLU benchmark the results are saturated – the best models of five providers are within 4 points of the average, and only the smaller models (Mistral Small 3.1, Gemma 3 27B) fall clearly behind.

4.12 Q12: Analysis of model names with string functions

The query (Listing 4.13) uses `REGEX`, `REPLACE` with a capturing group, `STRBEFORE`, `STRAFTER` and `UCASE` to split names at their first version number into a model family and a version label (for example “Claude Sonnet 4.5” into “CLAUDE SONNET” and “4.5”), and then aggregates the releases per family with `HAVING (COUNT(?m) > 1)`.

Listing 4.13
SPARQL query Q12

```
SELECT ?family (COUNT(?m) AS ?releases)
       (GROUP_CONCAT(?version; separator=" -> ") AS ?versions)
       (MIN(?released) AS ?firstRelease) (MAX(?released) AS ?lastRelease)
WHERE {
  {
    SELECT ?m ?released ?version ?family
    WHERE {
      ?m a/rdfs:subClassOf* ait:AIModel ;
         rdfs:label ?name ;
         ait:releaseDate ?released .
      FILTER (REGEX(?name, "[0-9]"))
      # first version number in the name, e.g. "4.5" in "Claude Sonnet 4.5"
      BIND (REPLACE(?name, "^[^0-9]*?([0-9]+(\\.[0-9]+)?).*$", "$1") AS ?number)
      # everything before it, e.g. "Claude Sonnet " or "Midjourney V"
      BIND (STRBEFORE(?name, ?number) AS ?prefix)
      # the rest of the name is the version label, e.g. "4o", "2.5 Pro", "6.1"
      BIND (STRAFTER(?name, ?prefix) AS ?version)
      BIND (UCASE(REPLACE(?prefix, "[\\s-]*[Vv]?[\\s-]*$", "")) AS ?rawFamily)
      BIND (IF(?rawFamily = "", "O-SERIES", ?rawFamily) AS ?family)
    }
    ORDER BY ?released
```

```
  }
}
GROUP BY ?family
HAVING (COUNT(?m) > 1)
ORDER BY DESC(?releases) ?family
```

Table 4.12
Results of query Q12

family	releases	versions	firstRelease	lastRelease
GEMINI	3	2.0 Flash -> 2.5 Pro -> 2.5 Flash	2024-12-11	2025-04-17
GPT	3	4o -> 4.1 -> 5	2024-05-13	2025-08-07
CLAUDE	2	3.5 Sonnet -> 3.7 Sonnet	2024-06-20	2025-02-24
CLAUDE SONNET	2	4 -> 4.5	2025-05-22	2025-09-29
LLAMA	2	3.1 405B -> 4 Maverick	2024-07-23	2025-04-05
MIDJOURNEY	2	6.1 -> 7	2024-07-30	2025-04-03

Note: The query returns 6 rows.

The query recovers model families purely from naming conventions (Table 4.12), which also shows the limits of this approach:

- The version label is everything after the family name, so it keeps letter suffixes and variant names: GPT-4o appears as “4o”, not as “4”, and the Gemini family contains two models with the version number 2.5 that differ only in the variant (“2.5 Pro” and “2.5 Flash”). The same version number therefore does not imply the same model.
- Anthropic changed its naming scheme from “Claude 3.7 Sonnet” to “Claude Sonnet 4”, so one product line is split into the two families CLAUDE and CLAUDE SONNET.

The explicit relation `ait:derivedFrom` (CQ7) is therefore the reliable way to express lineage; string analysis is useful only as a quick, approximate overview.

4.13 Q13: Competitor network

The relation `competesWith` is declared symmetric but is stated in only one direction in the data; the alternative path `ait:competesWith|^ait:competesWith` follows both directions without reasoning (Listing 4.14). `EXISTS` inside `IF` tests whether two competitors share an underlying model.

Listing 4.14
SPARQL query Q13

```
SELECT ?tool (COUNT(?c) AS ?competitors)
       (SUM(?shares) AS ?sharingModel)
       (GROUP_CONCAT(?competitor; separator=", ") AS ?competitorNames)
WHERE {
  {
    SELECT ?tool ?c ?competitor ?shares
    WHERE {
```

```
      ?t a ait:AITool ; rdfs:label ?tool .
      OPTIONAL {
        ?t ait:competesWith|^ait:competesWith ?c .
        ?c rdfs:label ?competitorName .
      }
      BIND (COALESCE(?competitorName, "-") AS ?competitor)
      # 1 if the tool and its competitor use at least one common model, otherwise 0
      BIND (IF(BOUND(?c) && EXISTS { ?t ait:poweredBy ?x . ?c ait:poweredBy ?x . }, 1,
0) AS ?shares)
    }
    ORDER BY ?competitor
  }
}
GROUP BY ?tool
ORDER BY DESC(?competitors) ?tool
```

Table 4.13
Results of query Q13

tool	competitors	sharingModel	competitorNames
ChatGPT	5	1	Claude, DeepSeek Chat, Gemini, Le Chat, Perplexity
Gemini	4	1	ChatGPT, Claude, Meta AI, Perplexity
GitHub Copilot	4	4	Claude Code, Cursor, Gemini Code Assist, OpenAI Codex
Claude	3	0	ChatGPT, Gemini, Le Chat
Claude Code	3	2	Cursor, GitHub Copilot, OpenAI Codex
Cursor	3	3	Claude Code, GitHub Copilot, OpenAI Codex
OpenAI Codex	3	2	Claude Code, Cursor, GitHub Copilot
DeepSeek Chat	2	0	ChatGPT, Qwen Chat
Le Chat	2	0	ChatGPT, Claude
Perplexity	2	2	ChatGPT, Gemini
Gemini Code Assist	1	1	GitHub Copilot
Meta AI	1	0	Gemini
Qwen Chat	1	0	DeepSeek Chat
Midjourney	0	0	-
Ollama	0	0	-

Note: The query returns 15 rows.

ChatGPT has the most direct competitors (Table 4.13). In the coding segment (GitHub Copilot, Cursor, Claude Code, OpenAI Codex) competitors almost always share models – they compete on the product rather than on the model. In the chat segment competitors mostly use their own models; Claude, Le Chat and DeepSeek Chat share no model with any of their competitors.

4.14 Q14: Models used in tools but lacking benchmark results

`FILTER NOT EXISTS` finds language models that are used by at least one tool but have no `BenchmarkResult` (Listing 4.15). This is a data-quality check that identifies gaps in the graph.

Listing 4.15
SPARQL query Q14

```
SELECT ?model ?developer (COUNT(DISTINCT ?t) AS ?usedInTools)
WHERE {
  ?m a ait:LanguageModel ;
     rdfs:label ?model ;
     ait:developedBy/rdfs:label ?developer .
  ?t ait:poweredBy ?m .
  FILTER NOT EXISTS { ?r ait:evaluatedModel ?m }
}
GROUP BY ?model ?developer
ORDER BY DESC(?usedInTools) ?model
```

Table 4.14
Results of query Q14

model	developer	usedInTools
Gemini 2.5 Flash	Google	1
Mistral Medium 3	Mistral AI	1

Note: The query returns 2 rows.

Two models used in production tools lack benchmark data in the graph (Table 4.14) and are candidates for the next data update.

4.15 Q15: Materialising tool dependencies with CONSTRUCT

The `CONSTRUCT` query (Listing 4.16) derives new knowledge: a direct link `schema:isBasedOn` from a tool to every external provider whose model it uses. The resulting graph (Listing 4.17, stored in `results/q15-construct-tool-dependency.ttl`) could be added to the knowledge graph to answer dependency questions without the two-step path.

Listing 4.16
SPARQL query Q15

```
PREFIX schema: <https://schema.org/>
CONSTRUCT {
  ?t schema:isBasedOn ?mp .
}
WHERE {
  ?t ait:providedBy ?tp ;
     ait:poweredBy/ait:developedBy ?mp .
  FILTER (?tp != ?mp)
}
```

Listing 4.17
Triples constructed by query Q15

```
aid:Cursor schema:isBasedOn aid:Anthropic,
        aid:Google,
        aid:OpenAI .

aid:GitHubCopilot schema:isBasedOn aid:Anthropic,
        aid:Google,
        aid:OpenAI .

aid:Ollama schema:isBasedOn aid:Alibaba,
        aid:DeepSeek,
        aid:Google,
        aid:Meta,
        aid:MistralAI,
        aid:OpenAI .

aid:Perplexity schema:isBasedOn aid:Anthropic,
        aid:Google,
        aid:OpenAI .
```

5 SHACL validation

The shapes (Knublauch & Kontokostas, 2017) are stored in `shacl/shapes.ttl` and were validated with pySHACL. They are organised into ten groups S1–S10 consisting of eleven node shapes; together they target every class of the ontology (`ait:OpenLicense` is covered through its superclass `ait:License`). Shape S10 uses a SPARQL-based constraint.

5.1 S1 AIToolShape

Every tool has a name of type `xsd:string`, exactly one provider (`sh:minCount 1` and `sh:maxCount 1`, value of class `ait:Provider`), at least one model of class `ait:AIModel`, at least one task and at least one pricing plan (Listing 5.1).

Listing 5.1
SHACL shape AIToolShape

```
aish:AIToolShape
    a sh:NodeShape ;
    sh:targetClass ait:AITool ;
    sh:property [
        sh:path rdfs:label ;
        sh:minCount 1 ;
        sh:datatype xsd:string ;
        sh:message "An AI tool must have a name (rdfs:label of type xsd:string)." ;
    ] , [
        sh:path ait:providedBy ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:class ait:Provider ;
        sh:message "An AI tool must have exactly one provider of type ait:Provider." ;
    ] , [
        sh:path ait:poweredBy ;
        sh:minCount 1 ;
        sh:class ait:AIModel ;
        sh:message "An AI tool must be powered by at least one ait:AIModel." ;
    ] , [
        sh:path ait:supportsTask ;
        sh:minCount 1 ;
        sh:class ait:Task ;
        sh:message "An AI tool must support at least one ait:Task." ;
    ] , [
        sh:path ait:hasPricingPlan ;
        sh:minCount 1 ;
        sh:class ait:PricingPlan ;
        sh:message "An AI tool must have at least one pricing plan." ;
    ] .
```

5.2 S2 AIModelShape

Every model has exactly one developer, exactly one release date of type `xsd:date`, exactly one licence and at least one input and one output modality. The context window and the

parameter count are optional, but when present they must be single positive numbers of the right datatype, expressed with `sh:minExclusive 0` (Listing 5.2).

Listing 5.2
SHACL shape AIModelShape

```
aish:AIModelShape
    a sh:NodeShape ;
    sh:targetClass ait:AIModel ;
    sh:property [
        sh:path ait:developedBy ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:class ait:Provider ;
        sh:message "An AI model must be developed by exactly one ait:Provider." ;
    ] , [
        sh:path ait:releaseDate ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:datatype xsd:date ;
        sh:message "An AI model must have exactly one release date of type
xsd:date." ;
    ] , [
        sh:path ait:hasLicense ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:class ait:License ;
        sh:message "An AI model must have exactly one licence." ;
    ] , [
        sh:path ait:inputModality ;
        sh:minCount 1 ;
        sh:class ait:Modality ;
        sh:message "An AI model must declare at least one input modality." ;
    ] , [
        sh:path ait:outputModality ;
        sh:minCount 1 ;
        sh:class ait:Modality ;
        sh:message "An AI model must declare at least one output modality." ;
    ] , [
        sh:path ait:contextWindow ;
        sh:maxCount 1 ;
        sh:datatype xsd:integer ;
        sh:minExclusive 0 ;
        sh:message "The context window must be a single positive integer (number of
tokens)." ;
    ] , [
        sh:path ait:parameterCount ;
        sh:maxCount 1 ;
        sh:datatype xsd:decimal ;
        sh:minExclusive 0 ;
        sh:message "The parameter count must be a single positive decimal (billions of
parameters)." ;
    ] .
```

5.3 S3 LanguageModelShape and ImageGenerationModelShape

These shapes add subclass-specific rules: a language model must state its context window and must output text (`sh:hasValue aid:Text`), and an image generation model must output images (Listing 5.3 and Listing 5.4).

Listing 5.3
SHACL shape LanguageModelShape

```
aish:LanguageModelShape
    a sh:NodeShape ;
    sh:targetClass ait:LanguageModel ;
    sh:property [
        sh:path ait:contextWindow ;
```

```
        sh:minCount 1 ;
        sh:message "A language model must state its context window." ;
    ] , [
        sh:path ait:outputModality ;
        sh:hasValue aid:Text ;
        sh:message "A language model must produce text output." ;
    ] .
```

Listing 5.4
SHACL shape ImageGenerationModelShape

```
aish:ImageGenerationModelShape
    a sh:NodeShape ;
    sh:targetClass ait:ImageGenerationModel ;
    sh:property [
        sh:path ait:outputModality ;
        sh:hasValue aid:Image ;
        sh:message "An image generation model must produce image output." ;
    ] .
```

5.4 S4 BenchmarkResultShape

Every result refers to exactly one model and one benchmark and has exactly one decimal score in the interval [0, 100], expressed with `sh:minInclusive` and `sh:maxInclusive` (Listing 5.5).

Listing 5.5
SHACL shape BenchmarkResultShape

```
aish:BenchmarkResultShape
    a sh:NodeShape ;
    sh:targetClass ait:BenchmarkResult ;
    sh:property [
        sh:path ait:evaluatedModel ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:class ait:AIModel ;
        sh:message "A benchmark result must refer to exactly one evaluated AI
model." ;
    ] , [
        sh:path ait:onBenchmark ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:class ait:Benchmark ;
        sh:message "A benchmark result must refer to exactly one benchmark." ;
    ] , [
        sh:path ait:score ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:datatype xsd:decimal ;
        sh:minInclusive 0 ;
        sh:maxInclusive 100 ;
        sh:message "A benchmark score must be a single decimal between 0 and 100." ;
    ] .
```

5.5 S5 PricingPlanShape

Every plan has a name and exactly one non-negative decimal price, and the currency must be one of the supported codes, expressed with `sh:in ("USD" "EUR")` (Listing 5.6).

Listing 5.6
SHACL shape PricingPlanShape

```
aish:PricingPlanShape
    a sh:NodeShape ;
```

```
    sh:targetClass ait:PricingPlan ;
    sh:property [
        sh:path rdfs:label ;
        sh:minCount 1 ;
        sh:message "A pricing plan must have a name." ;
    ] , [
        sh:path ait:monthlyPrice ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:datatype xsd:decimal ;
        sh:minInclusive 0 ;
        sh:message "A pricing plan must have exactly one non-negative monthly price
(xsd:decimal)." ;
    ] , [
        sh:path ait:currency ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:in ( "USD" "EUR" ) ;
        sh:message "The currency must be one of the supported ISO 4217 codes: USD,
EUR." ;
    ] .
```

5.6 S6 ProviderShape

Every provider has a name and a headquarters country; the founding year, if stated, must be a single value of type `xsd:gYear`, and the homepage must be an IRI rather than a string literal (`sh:nodeKind sh:IRI`) (Listing 5.7).

Listing 5.7
SHACL shape ProviderShape

```
aish:ProviderShape
    a sh:NodeShape ;
    sh:targetClass ait:Provider ;
    sh:property [
        sh:path rdfs:label ;
        sh:minCount 1 ;
        sh:datatype xsd:string ;
        sh:message "A provider must have a name (rdfs:label of type xsd:string)." ;
    ] , [
        sh:path ait:foundedYear ;
        sh:maxCount 1 ;
        sh:datatype xsd:gYear ;
        sh:message "The founding year must be a single value of type xsd:gYear." ;
    ] , [
        sh:path ait:headquartersCountry ;
        sh:minCount 1 ;
        sh:datatype xsd:string ;
        sh:message "A provider must state its headquarters country." ;
    ] , [
        sh:path foaf:homepage ;
        sh:nodeKind sh:IRI ;
        sh:message "A homepage must be an IRI, not a literal." ;
    ] .
```

5.7 S7 LicenseShape

Every licence has exactly one SPDX identifier. The regular expression in `sh:pattern` accepts identifiers from the SPDX License List (for example `Apache-2.0`) as well as custom identifiers with the prefix `LicenseRef-`, which SPDX prescribes for licences that are not on the list (Listing 5.8).

Listing 5.8
SHACL shape LicenseShape

```
aish:LicenseShape
    a sh:NodeShape ;
    sh:targetClass ait:License ;
    sh:property [
        sh:path ait:spdxId ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:datatype xsd:string ;
        sh:pattern "^(LicenseRef-[A-Za-z0-9.-]+|[A-Za-z0-9][A-Za-z0-9.+-]*)$" ;
        sh:message "A licence must have exactly one valid SPDX identifier (e.g.
'Apache-2.0' or 'LicenseRef-Proprietary')." ;
    ] .
```

5.8 S8 BenchmarkShape

Every benchmark measures at least one task and states exactly one metric in which its scores are expressed (Listing 5.9).

Listing 5.9
SHACL shape BenchmarkShape

```
aish:BenchmarkShape
    a sh:NodeShape ;
    sh:targetClass ait:Benchmark ;
    sh:property [
        sh:path ait:measuresTask ;
        sh:minCount 1 ;
        sh:class ait:Task ;
        sh:message "A benchmark must measure at least one ait:Task." ;
    ] , [
        sh:path ait:metric ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:datatype xsd:string ;
        sh:message "A benchmark must state exactly one metric." ;
    ] .
```

5.9 S9 VocabularyShape

Tasks, modalities and integrations are controlled vocabularies whose instances are shown to users by name. One shape with three target classes requires that each of them has exactly one name (Listing 5.10).

Listing 5.10
SHACL shape VocabularyShape

```
aish:VocabularyShape
    a sh:NodeShape ;
    sh:targetClass ait:Task , ait:Modality , ait:Integration ;
    sh:property [
        sh:path rdfs:label ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:datatype xsd:string ;
        sh:message "A task, modality or integration must have exactly one name." ;
    ] .
```

5.10 S10 ModelLineageShape

A model cannot have been released before a model it is derived from. This temporal rule compares values of two different nodes and cannot be expressed with core SHACL constraints, so a `sh:SPARQLConstraint` is used; every row returned by the `SELECT` query is reported as a violation (Listing 5.11).

Listing 5.11
SHACL shape ModelLineageShape

```
aish:ModelLineageShape
    a sh:NodeShape ;
    sh:targetClass ait:AIModel ;
    sh:sparql [
        a sh:SPARQLConstraint ;
        sh:message "Model {$this} (released {?date}) is derived from {?parent}, which
was released later ({?parentDate})." ;
        sh:prefixes aish: ;
        sh:select """
            SELECT $this ?parent ?date ?parentDate
            WHERE {
                $this ait:derivedFrom ?parent ;
                      ait:releaseDate ?date .
                ?parent ait:releaseDate ?parentDate .
                FILTER (?date < ?parentDate)
            }
        """ ;
    ] .
```

5.11 Validation results

The valid knowledge graph (`data/ai-tools-data.ttl`) conforms to all shapes: **Conforms:** `true` — 0 violation(s).

To test the shapes, the knowledge graph was validated together with the file `data/ai-tools-data-invalid.ttl`. The file contains eleven broken resources: a tool without a provider and a pricing plan, a tool "powered by" a provider instead of a model, a model with a string release date, a negative context window, no licence and audio-only output, an image model producing only text, a benchmark result with a score of 112.5 and no benchmark, a plan with a negative price in CZK, a model released before its predecessor, a provider with an integer founding year, no country and a string homepage, a licence with an invalid SPDX identifier, a benchmark without a task and a metric, and a task with two names. The violations found are listed in Table 5.1.

Table 5.1
SHACL violations found in the intentionally invalid data

Focus node	Shape	Path	Constraint	Message
BadBenchmark	S8	ait:measuresTask	sh:MinCount	A benchmark must measure at least one ait:Task.
BadBenchmark	S8	ait:metric	sh:MinCount	A benchmark must state exactly one metric.

Focus node	Shape	Path	Constraint	Message
BadLicense	S7	ait:spdxId	sh:Pattern	A licence must have exactly one valid SPDX identifier (e.g. ‘Apache-2.0’ or ‘LicenseRef-Proprietary’).
BadModel_NoImage	S3	ait:outputModality	sh:HasValue	An image generation model must produce image output.
BadModel_TimeTravel	S10	(SPARQL)	sh:SPARQL	Model aid:BadModel_TimeTravel (released 2024-01-01) is derived from aid:ClaudeSonnet45, which was released later (2025-09-29).
BadModel_Types	S2	ait:contextWindow	sh:MinExclusive	The context window must be a single positive integer (number of tokens).
BadModel_Types	S2	ait:hasLicense	sh:MinCount	An AI model must have exactly one licence.
BadModel_Types	S2	ait:releaseDate	sh:Datatype	An AI model must have exactly one release date of type xsd:date.
BadModel_Types	S3	ait:outputModality	sh:HasValue	A language model must produce text output.
BadPlan_Price	S5	ait:currency	sh:In	The currency must be one of the supported ISO 4217 codes: USD, EUR.
BadPlan_Price	S5	ait:monthlyPrice	sh:MinInclusive	A pricing plan must have exactly one non-negative monthly price (xsd:decimal).
BadProvider	S6	ait:foundedYear	sh:Datatype	The founding year must be a single value of type xsd:gYear.
BadProvider	S6	ait:headquartersCountry	sh:MinCount	A provider must state its headquarters country.
BadProvider	S6	foaf:homepage	sh:NodeKind	A homepage must be an IRI, not a literal.
BadResult_Score	S4	ait:onBenchmark	sh:MinCount	A benchmark result must refer to exactly one benchmark.
BadResult_Score	S4	ait:score	sh:MaxInclusive	A benchmark score must be a single decimal between 0 and 100.
BadTask	S9	rdfs:label	sh:MaxCount	A task, modality or integration must have exactly one name.
BadTool_NoProvider	S1	ait:hasPricingPlan	sh:MinCount	An AI tool must have at least one pricing plan.

Focus node	Shape	Path	Constraint	Message
BadTool_NoProvider	S1	ait:providedBy	sh:MinCount	An AI tool must have exactly one provider of type ait:Provider.
BadTool_WrongModel	S1	ait:poweredBy	sh:Class	An AI tool must be powered by at least one ait:AIModel.

Note: Conforms: false — 20 violation(s).

Every intentionally introduced error was detected by the expected shape and constraint component, and the valid data conform to all shapes. This shows that the shapes are neither too weak nor too strict for the real data.

Conclusion

The goal of the work – to design and build a knowledge graph of AI tools that answers practical selection and market-analysis questions – was achieved. We designed an ontology with 13 classes, 25 properties and a range of OWL axioms, populated it with a knowledge graph of almost 1,000 triples, answered ten competency questions and five additional analytical queries in SPARQL, and validated the data with eleven SHACL shapes that cover every class of the ontology.

The most valuable modelling decisions were the separation of tools from models and the modelling of benchmark results and pricing plans as separate nodes. They enable the most interesting analyses: dependencies between providers (CQ6, Q15), price comparisons (CQ3) and comparison of models with benchmark averages (Q11). The queries also showed, for example, that third-party tools such as GitHub Copilot, Cursor and Perplexity rely on the models of the same three providers, and that openly licensed models reach the level of closed models on older benchmarks.

Limitations. The data are a manually created snapshot as of 30 September 2025 and will age quickly; benchmark scores are mostly vendor-reported under different settings, and some conclusions (for example about release intervals in CQ7) are based on only a few data points.

Future work. A natural next step would be to populate the graph automatically from structured sources (for example Hugging Face model cards, Wikidata and public leaderboards) using YARRRML or RML mappings, to track prices and benchmark results over time, to add API pricing per token and to extend the `Task` concept into a SKOS hierarchy (for example *code completion* as a narrower concept of *code generation*).

References

Beckett, D., Berners-Lee, T., Prud'hommeaux, E., & Carothers, G. (2014). *RDF 1.1 Turtle: Terse RDF triple language* (W3C Recommendation). World Wide Web Consortium. https://www.w3.org/TR/turtle/

Brickley, D., & Miller, L. (2014). *FOAF vocabulary specification 0.99*. http://xmlns.com/foaf/spec/

DCMI Usage Board. (2020). *DCMI metadata terms*. Dublin Core Metadata Initiative. https://www.dublincore.org/specifications/dublin-core/dcmi-terms/

Grüninger, M., & Fox, M. S. (1995). Methodology for the design and evaluation of ontologies. In *Proceedings of the Workshop on Basic Ontological Issues in Knowledge Sharing, IJCAI-95*. Montreal, Canada.

Guha, R. V., Brickley, D., & Macbeth, S. (2016). Schema.org: Evolution of structured data on the web. *Communications of the ACM, 59*(2), 44–51. https://doi.org/10.1145/2844544

Harris, S., & Seaborne, A. (Eds.). (2013). *SPARQL 1.1 query language* (W3C Recommendation). World Wide Web Consortium. https://www.w3.org/TR/sparql11-query/

Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J. E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., & Zimmermann, A. (2021). Knowledge graphs. *ACM Computing Surveys, 54*(4), Article 71. https://doi.org/10.1145/3447772

Knublauch, H., & Kontokostas, D. (Eds.). (2017). *Shapes Constraint Language (SHACL)* (W3C Recommendation). World Wide Web Consortium. https://www.w3.org/TR/shacl/

Miles, A., & Bechhofer, S. (Eds.). (2009). *SKOS Simple Knowledge Organization System reference* (W3C Recommendation). World Wide Web Consortium. https://www.w3.org/TR/skos-reference/

Noy, N., & Rector, A. (Eds.). (2006). *Defining n-ary relations on the Semantic Web* (W3C Working Group Note). World Wide Web Consortium. https://www.w3.org/TR/swbp-n-aryRelations/

Noy, N. F., & McGuinness, D. L. (2001). *Ontology development 101: A guide to creating your first ontology* (Technical Report KSL-01-05). Stanford Knowledge Systems Laboratory.

Publio, G. C., Esteves, D., Ławrynowicz, A., Panov, P., Soldatova, L., Soru, T., Vanschoren, J., & Zafar, H. (2018). *ML-Schema: Exposing the semantics of machine learning with schemas and ontologies*. arXiv. https://doi.org/10.48550/arXiv.1807.05351

SPDX Workgroup. (n.d.). *SPDX license list*. The Linux Foundation. Retrieved November 10, 2026, from https://spdx.org/licenses/

Vandenbussche, P.-Y., Atemezing, G. A., Poveda-Villalón, M., & Vatant, B. (2017). Linked Open Vocabularies (LOV): A gateway to reusable semantic vocabularies on the Web. *Semantic Web, 8*(3), 437–452. https://doi.org/10.3233/SW-160213

W3C OWL Working Group. (2012). *OWL 2 Web Ontology Language document overview (second edition)* (W3C Recommendation). World Wide Web Consortium. https://www.w3.org/TR/owl2-overview/

Appendices

Appendix A: Repository and reproducibility

All artefacts of the work are available in the public repository https://github.com/otavlna/4iz444-ai-tools-kg. Its structure is described in Table A.1.

Table A.1
Structure of the repository

Path	Content
`ontology/ai-tools.ttl`	ontology (OWL 2, Turtle)
`data/ai-tools-data.ttl`	knowledge graph
`data/ai-tools-data-invalid.ttl`	intentionally invalid data for testing the SHACL shapes
`queries/*.rq`	SPARQL queries CQ1–CQ10 and Q11–Q15
`shacl/shapes.ttl`	SHACL shapes S1–S10
`results/`	query results (CSV, Markdown, Turtle) and SHACL reports
`scripts/build.py`	build pipeline: parsing, documentation check, reasoning, queries, SHACL, diagram
`scripts/build_docx.py`	conversion of the report into the FIS Word template
`report/`	source of the report, the template and the resulting DOCX and PDF

Running `make all` re-creates all results, the diagram and this report from the source files.

Appendix B: Use of artificial intelligence tools

Table B.1 documents the use of AI tools in individual parts of the work, as required by the declaration at the beginning of the document.

Table B.1
Use of AI tools in individual parts of the work

Part of the work	Tool	How the tool was used	Our own contribution
Choice of domain and core classes	–	–	Domain and initial list of classes proposed by Matěj Fadrhons.
Ontology	Claude Code (Anthropic)	Drafting classes, properties, axioms and documentation from our list of classes.	Review of the design decisions.

Part of the work	Tool	How the tool was used	Our own contribution
Data	Claude Code (Anthropic)	Generating the instance data from public information about tools, models, prices and benchmarks.	Review of the data, decision on the snapshot date.
SPARQL queries and SHACL shapes	Claude Code (Anthropic)	Drafting the queries for the competency questions and the shapes.	Review of the competency questions and of the query results.
Build scripts	Claude Code (Anthropic)	Generating the source code of the build pipeline.	Running and checking the pipeline.
Text of the report	Claude Code (Anthropic)	Drafting and language editing of the text.	Review and corrections of the text.
Review of the work	Codex (OpenAI)	Automated review of the report and the data, which identified inaccuracies.	Evaluation of the findings and their correction.

Appendix C: Contribution of team members

The contribution of the individual team members is summarised in Table C.1.

Table C.1
Contribution of team members

Team member	Contribution
Ota Vlna	Coordination of the work; generation and review of the ontology, data, queries and shapes; repository and build pipeline; text of the report.
Matěj Fadrhons	Choice of the domain and of the core classes; review of the report and the data.